

DAKKHO Platform

Full Architecture Document

Admin Panel + Student Panel + Cloudflare Workers + Appwrite

Item	Details
Project	DAKKHO Learning Platform
Backend	Cloudflare Workers (Hono Framework)
Frontend	Next.js 16 + React 19 + Zustand + shadcn/ui
Database	Appwrite (dakkho_main) + D1 (sessions/audit)
Storage	R2 (4 buckets: videos/thumbnails/avatars/resources)
Auth	Appwrite Auth + D1 Session Tokens
Email	Resend API
Date	2026-06-04

1. System Overview

DAKKHO is a full-stack web application built with Next.js 16, Cloudflare Pages, Cloudflare Workers, Cloudflare KV, Cloudflare D1, Cloudflare R2, and Appwrite. The application is hosted on Cloudflare Pages and Cloudflare Workers. The frontend is built with Next.js 16 and Cloudflare Pages. The backend is built with Cloudflare Workers and Cloudflare KV. The database is Cloudflare D1. The storage is Cloudflare R2. The email service is Resend.

The application consists of two main parts: the Admin Panel and the Student Panel. The Admin Panel is a SPA (Single Page Application) built with Next.js and Zustand. The Student Panel is a SPA built with Next.js and Zustand. The Admin Panel uses the Workers API for authentication. The Student Panel uses the Workers API for authentication. The Admin Panel uses the Workers API for authentication. The Student Panel uses the Workers API for authentication. The Admin Panel uses the Workers API for authentication. The Student Panel uses the Workers API for authentication.

Architecture Diagram (Text)

Layer	Component	Technology	Role
Frontend	Admin Panel	Next.js + Zustand + shadcn/ui	Admin dashboard SPA
Frontend	Student Panel	Next.js + Zustand + shadcn/ui	Student learning SPA
CDN/Hosting	Cloudflare Pages	Static site hosting	Frontend delivery
API	Cloudflare Workers	Hono framework	REST API backend
Database	Appwrite	Cloud (SGP region)	Primary data + auth
Database	D1	Cloudflare native	Sessions + audit + config
Storage	R2	4 buckets	Videos/thumbnails/avatars/resources
Cache	Workers KV	Cloudflare native	Config cache + broadcast
Email	Resend	REST API	Transactional email

2. Authentication Flow

2.1 Admin Auth (Current)

Admin `POST /admin/auth/login` email password Worker Appwrite-
 prefs.role D1-
 Appwrite Worker D1
 auth localStorage- token
 (sessionId) API
 Authorization: Bearer header

Step	Action	Detail
1	Appwrite Login	POST /account/sessions/email with email+password, get sessionCookie
2	Get Account	GET /account with sessionCookie, verify prefs.role === "admin"
3	Create D1 Session	DELETE old sessions for user_id, INSERT new session (id=UUID, expires_at+=7d)
4	Delete Appwrite Session	DELETE /account/sessions/current (we use our own token model)
5	Return Token	{ success: true, token: sessionId, user: {id, email, name, role} }
6	Frontend Stores	localStorage.setItem("dakkho_admin_token", sessionId)

2.2 Student Auth (Recommended)

Student Panel- auth model
 D1 Appwrite Auth Worker Appwrite
 Student API
 user-scoped Appwrite queries
 Appwrite permission enforce

Aspect	Admin Panel	Student Panel
Auth Provider	D1 Session Token	Appwrite Session Cookie
Token Storage	localStorage	httpOnly Cookie or localStorage
Token Format	UUID (sessionId)	Appwrite session token
Auth Header	Authorization: Bearer {id}	Cookie: a_session_dakkho={token}

Permission Model	API key (full access)	User-scoped (own data only)
Session Store	D1 admin_sessions table	Appwrite internal
Expiry	7 days	Appwrite default (1 year)

3. API Endpoints Reference

API endpoint Cloudflare Workers-<https://dakkho-admin-api.dakkho-admin.workers.dev/>

Admin routes /admin/ prefix-[Bearer token auth](#) Student routes /student/ prefix-[Appwrite session auth](#)
[CORS](#) [origins](#) (Cloudflare Pages, localhost)
[allow](#)

3.1 Admin API (Current)

Method	Endpoint	Auth	Description
POST	/admin/auth/login	No	Login with email+password, returns token+user
POST	/admin/auth/check	Bearer	Verify if session is valid
DELETE	/admin/auth/logout	Bearer	Invalidate admin session
GET	/admin/users	Bearer	List users (page, limit, search, role, status)
PUT	/admin/users	Bearer	Update user (userId + updates)
DELETE	/admin/users	Bearer	Delete user by id
GET	/admin/courses	Bearer	List courses (page, limit, search, level, published, featured)
POST	/admin/courses	Bearer	Create course
PUT	/admin/courses	Bearer	Update course (courseId + updates)
DELETE	/admin/courses	Bearer	Delete course by id
GET	/admin/videos	Bearer	List videos (page, limit, courseId, published)

POST	/admin/videos	Bearer	Create video
PUT	/admin/videos	Bearer	Update video
DELETE	/admin/videos	Bearer	Delete video
GET	/admin/instructors	Bearer	List instructors (page, limit, search)
GET	/admin/categories	Bearer	List categories (limit, orderAsc by sort_order)
GET	/admin/institutes	Bearer	List institutes
GET	/admin/config	Bearer	Get full ServerConfig (KV cache then D1)
PUT	/admin/config	Bearer	Update ServerConfig (D1 upsert + KV update)
GET	/admin/notifications	Bearer	List notifications (page, limit, userId)
POST	/admin/notifications	Bearer	Create notification (targetAll/targetUserId/targetInstitute)
GET	/admin/analytics	Bearer	Stats: users, courses, videos, enrollments, sessions
POST	/admin/upload	Bearer	Upload file to R2 (FormData: file, bucket, prefix)
POST	/admin/email/test	Bearer	Send test email via Resend
GET	/admin/system/status	No	Health check: appwrite, r2, d1, kv, email
POST	/admin/system/api-key	Bearer	Update Appwrite API key in KV

3.2 Student API (Recommended — New)

Student Panel-🔒 🔒🔒🔒 🔒🔒🔒 /student/ route group 🔒🔒🔒 🔒🔒🔒
🔒🔒🔒 Worker-🔒🔒 🔒🔒 routes Appwrite session cookie 🔒🔒🔒🔒🔒 auth 🔒🔒🔒🔒, API
key 🔒🔒🔒🔒🔒 🔒🔒🔒🔒 🔒🔒🔒🔒🔒🔒🔒🔒 🔒🔒🔒🔒 🔒🔒🔒🔒🔒 🔒🔒🔒🔒🔒🔒
🔒🔒🔒🔒🔒, watch progress, bookmarks, notifications 🔒🔒🔒🔒🔒 🔒🔒🔒🔒🔒🔒
🔒🔒🔒🔒 recommended endpoints 🔒🔒🔒🔒🔒🔒 🔒🔒🔒:

Method	Endpoint	Description
--------	----------	-------------

POST	/student/auth/login	Login with email+password, returns Appwrite session cookie
POST	/student/auth/check	Verify Appwrite session, return user profile
DELETE	/student/auth/logout	Delete Appwrite session
GET	/student/profile	Get own profile from Appwrite users collection
PUT	/student/profile	Update own profile (name, avatarUrl, etc.)
GET	/student/courses	List enrolled courses with progress
GET	/student/courses/all	List all published courses (catalog)
POST	/student/enroll	Enroll in a course (create enrollment doc)
GET	/student/videos	List videos for enrolled course (courseId required)
GET	/student/watch-progress	Get watch progress (videoId, courseId)
POST	/student/watch-progress	Update watch progress (position, completed)
GET	/student/bookmarks	List bookmarked courses
POST	/student/bookmarks	Add course bookmark
DELETE	/student/bookmarks	Remove bookmark (courseId)
GET	/student/notifications	List own notifications (unread first)
PUT	/student/notifications/read	Mark notification(s) as read
GET	/student/discussions	List discussions for a course
POST	/student/discussions	Create discussion/reply
GET	/student/settings	Get user settings
PUT	/student/settings	Update user settings
GET	/student/categories	List all categories (public)
GET	/student/instructors	List instructors (public)
GET	/student/config	Get ServerConfig (feature toggles, nav, style, protection)

4. Appwrite Collections Schema

Appwrite project: dakkho, Database: dakkho_main, Endpoint:
<https://sgp.cloud.appwrite.io/v1> `collection Admin`
Student Panel `collection-`
Appwrite-generated fields (\$id, \$createdAt, \$updatedAt, \$permissions)

Student Panel- documents- read permission "any"
user-specific

Collection	Key Fields	Student Use
users	fullName, email, institute, technology, avatarUrl, role, emailVerified, isActive, enrolledCourseIds	Profile + enrollment
courses	title, slug, description, thumbnailUrl, previewVideoUrl, categoryId, instructorId, level, language, duration, totalVideos, rating, totalReviews, totalStudents, isFeatured, isPublished, tags	Catalog + enrolled
videos	title, slug, description, courseId, videoUrl, thumbnailUrl, duration, order, isPreview, isPublished	Watch + progress
instructors	name, email, bio, avatarUrl, coverUrl, specialization, rating, totalStudents, totalCourses	Public profiles
institutes	name, code, address	Filter users
categories	name, slug, icon, color, parentId, order, courseCount	Course catalog
enrollments	userId, courseId, progress, completed	Track enrollment + progress
notifications	userId, title, message, type, isRead, actionUrl	In-app notifications
discussions	title, body, courseId, authorId, tags, isAnswered, repliesCount	Q&A; forum
user_settings	userId, streamingQuality, downloadQuality, autoDownload, wifiOnly, dataSaverMode, pushNotifications, emailNotifications, themeMode, appLanguage, profileVisibility	Personalization
bookmarks	userId, courseId	Save courses
watch_progress	userId, videoId, courseId, progress, lastPosition, completed	Resume watching

Enum Values

Field	Values
users.role	student instructor admin
courses.level	beginner intermediate advanced expert
courses.language	bangla english hindi
notifications.type	info success warning error announcement course-update

5. D1 Database (Cloudflare Native)

D1 `admin_sessions` Admin Panel-`app_config` Student Panel-`D1` `appwrite` `user_auth` `document_storage` `admin_sessions` `app_config` `audit_logs`: `admin_sessions` (auth), `app_config` (server-driven config), `audit_logs` (admin activity log)

Table	Columns	Purpose
<code>admin_sessions</code>	<code>id</code> (PK UUID), <code>user_id</code> (UNIQUE), <code>email</code> , <code>name</code> , <code>role</code> , <code>ip_address</code> , <code>user_agent</code> , <code>created_at</code> , <code>expires_at</code> , <code>is_active</code>	Admin session tokens (7-day expiry, single session per user)
<code>app_config</code>	<code>key</code> (PK), <code>value</code> (JSON), <code>description</code> , <code>updated_by</code> , <code>updated_at</code> , <code>created_at</code>	Server-driven config cache (KV fallback, seeded with 4 default keys)
<code>audit_logs</code>	<code>id</code> (PK UUID), <code>action</code> , <code>resource_type</code> , <code>resource_id</code> , <code>user_id</code> , <code>user_email</code> , <code>details</code> (JSON), <code>ip_address</code> , <code>created_at</code>	Admin activity tracking (non-blocking, never fails requests)

6. Frontend Architecture

6.1 API Client Pattern

`api-client.ts` API `Remote` (Workers) `Local` (Next.js API routes) `Remote` mode-`NEXT_PUBLIC_API_BASE_URL` env var `Workers` `local` Next.js routes-`URL` construction pattern: `buildUrl("/users?limit=20")` returns "`https://dakkho-admin-api.xxx.workers.dev/admin/users?limit=20`" `Auth` header `inject` `if IS_REMOTE_API` then `headers["Authorization"] = "Bearer " + localStorage.token`

Function	Signature	Use Case
<code>apiGet</code>	<code>apiGet(path): Promise</code>	Read data (GET)
<code>apiPost</code>	<code>apiPost(path, body): Promise</code>	Create/login (POST + JSON body)
<code>apiPut</code>	<code>apiPut(path, body): Promise</code>	Update (PUT + JSON body)
<code>apiDelete</code>	<code>apiDelete(path): Promise</code>	Remove (DELETE)
<code>apiUpload</code>	<code>apiUpload(path, formData): Promise</code>	File upload (POST + FormData, no Content-Type)
<code>apiRaw</code>	<code>apiRaw(path, init?): Promise</code>	Advanced use (raw Response)
<code>setAuthToken</code>	<code>setAuthToken(token): void</code>	Save token to localStorage after login

getAuthToken	getAuthToken(): string null	Read token from localStorage
clearAuthToken	clearAuthToken(): void	Remove token on logout

6.2 State Management (Zustand)

Admin Panel- Zustand store (useAdminStore) Student Panel- Zustand store (useStudentStore) Admin store- adminUser, serverConfig, sidebarCollapsed, currentPage, isLoading Student store- studentUser, enrolledCourses, watchProgress, notifications, settings Page routing single-page – currentPage state component render decide, Next.js router

6.3 Component Architecture

Admin Panel- 14 React component: LoginForm, Sidebar, Header, Dashboard, UsersTable, CoursesTable, VideosTable, InstructorsTable, CategoriesTable, InstitutesTable, NotificationsPanel, ConfigPanel, EmailPanel, AnalyticsPanel, SettingsPanel Student Panel- 10 React component: StudentLogin, CourseCatalog, CourseDetail, VideoPlayer, WatchProgress, BookmarksPanel, DiscussionForum, ProfileSettings, NotificationCenter component shadcn/ui + Tailwind CSS + Framer Motion, dark glassmorphism theme

7. Deployment Configuration

Component	Platform	URL / Detail
Admin Frontend	Cloudflare Pages	https://dakkho-admin.pages.dev/
Student Frontend	Cloudflare Pages	https://dakkho-student.pages.dev/ (to be created)
API Worker	Cloudflare Workers	https://dakkho-admin-api.dakkho-admin.workers.dev/
Code Repo	GitHub (Private)	github.com/grayrat2026/dakkho-admin
CI/CD	GitHub Actions	.github/workflows/deploy.yml
Build Command	Next.js static export	output: "export", NEXT_PUBLIC_API_BASE_URL set

Environment Variables

Variable	Admin Panel Value	Student Panel Value
NEXT_PUBLIC_API_BASE_URL	https://dakkho-admin-api.dakkho-admin.workers.dev	Same Worker URL
NEXT_PUBLIC_BASE_PATH	(empty, no basePath)	(empty, no basePath)
NEXT_PUBLIC_STATIC_MODE	(not needed)	(not needed)

8. Student Panel Build Prompt

Build a DAKKHO Student Panel — a Next.js 16 App Router SPA with static export (output: "export"), deployed on Cloudflare Pages. TECH STACK: Next.js 16 + React 19 + TypeScript + Zustand + shadcn/ui + Tailwind CSS 4 + Framer Motion. Dark glassmorphism theme (bg: #0F0F1A, glass-card: bg-white/5 backdrop-blur-xl border-white/10). BACKEND: Cloudflare Workers API at https://dakkho-admin-api.dakkho-admin.workers.dev/. You need to ADD /student/* route group to the existing Worker. APPWRITE: Project "dakkho", Database "dakkho_main", Endpoint https://sgp.cloud.appwrite.io/v1. 12 collections: users, courses, videos, instructors, institutes, categories, enrollments, notifications, discussions, user_settings, bookmarks, watch_progress. AUTH: Students login via Appwrite email session. Worker creates session, returns session cookie. Student API requests use Cookie: a_session_dakkho={token} header. NOT Bearer token like Admin. API CLIENT: Same pattern as Admin — apiGet/apiPost/apiPut/apiDelete/apiUpload functions. URL pattern: {BASE_URL}/student/{path}. Auth header: Cookie instead of Authorization Bearer. PAGES (single-page, Zustand currentPage routing):

1. Login — email/password form
 2. Home — featured courses, continue watching, categories
 3. Course Catalog — search/filter courses by category/level/language
 4. Course Detail — overview, curriculum (video list), instructor info, enroll button
 5. Video Player — video playback with progress tracking, next/prev navigation
 6. My Courses — enrolled courses with progress bars
 7. Bookmarks — saved courses
 8. Discussions — Q&A; forum per course
 9. Notifications — in-app notification center
 10. Profile — edit name/avatar/institute/technology
 11. Settings — streaming quality, download, notifications, theme, language
- COMPONENTS: Same glass-card style, gradient-primary buttons (bg-gradient-to-r from-blue-500 to-purple-500), dakkho-blue accent (#3B82F6). Sidebar navigation with collapse toggle. Responsive design.

DEPLOY: Cloudflare Pages, project name "dakkho-student". Same CI/CD as Admin (GitHub Actions). Build: next build (output: "export"), then wrangler pages deploy out/.

9. Key Implementation Notes

Topic	Detail
Appwrite Query Format	Appwrite v1.9+ requires JSON format queries: <code>{"method":"limit","values":[20]}</code> . Old string format like <code>limit(20)</code> does NOT work. Use <code>JSON.stringify()</code> in Query helpers.
CORS Configuration	Worker CORS must include all frontend origins: <code>dakkho-admin.pages.dev</code> , <code>dakkho-student.pages.dev</code> , <code>localhost:3000</code> . Allow credentials: <code>true</code> for cookies.
R2 Public URLs	Use <code>getPublicUrl(env, bucketType, key)</code> helper. Do NOT hardcode <code>.r2.dev</code> URLs. If <code>R2_PUBLIC_URL</code> env var is set, it takes precedence.
Config Broadcast	ServerConfig changes are written to D1 (source of truth) + KV (cache). Frontend polls <code>GET /student/config</code> periodically. KV key <code>"config_updated_at"</code> tracks last update timestamp.
Student Auth Middleware	New <code>studentAuthMiddleware</code> : validate Appwrite session cookie, get account info, set <code>c.set("studentUser", {...})</code> . Reject if not student role.
Error Handling	All Worker routes use <code>try/catch</code> with <code>getErrorMessage()</code> . Return <code>{ error: string }</code> on failure. Frontend catches <code>ApiError</code> with <code>status/code/message</code> .
Static Export	Next.js output: <code>"export"</code> means NO server-side features (no API routes, no ISR, no middleware). All data fetching via client-side API calls in <code>useEffect/hooks</code> .
Session Cookie	Appwrite session cookie name: <code>a_session_dakkho</code> . For Worker-to-Appwrite requests, send as Cookie header. Worker must pass this through from client request.